

React State Management: Context, Redux & TanStack Query

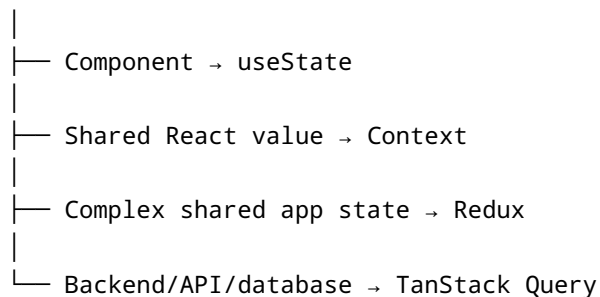
1. Big Picture

The easiest way to distinguish them:

Tool	Main Purpose	Example
<code>useState</code>	Local component state	Modal open/close
Context	Share values across components	Theme, language
Redux	Complex shared/client state	Cart, complex UI state
TanStack Query	Server/API state	Products, users, orders

Core question

Where does the data come from?



2. Context API

Purpose

Context prevents **prop drilling**.

Without Context:

```
App
  ↓ props
Dashboard
  ↓ props
```

Profile
↓ props
UserName

With Context:

```
      Context Provider
      /      |
      ↓      ↓      ↓
Dashboard Profile Settings
      |
      ↓
    useContext()
```

Basic flow

```
createContext()
  ↓
  Provider
  ↓
Components
  ↓
useContext()
  ↓
Read shared value
```

Example

```
const ThemeContext = createContext();

function App() {
  const [theme, setTheme] = useState("dark");

  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Dashboard />
    </ThemeContext.Provider>
  );
}

function Dashboard() {
  const { theme, setTheme } = useContext(ThemeContext);

  return (
```

```

    <button onClick={() => setTheme("light")}>
      {theme}
    </button>
  );
}

```

Remember

Context = a way to share values.

It does not automatically provide sophisticated state management or API caching.

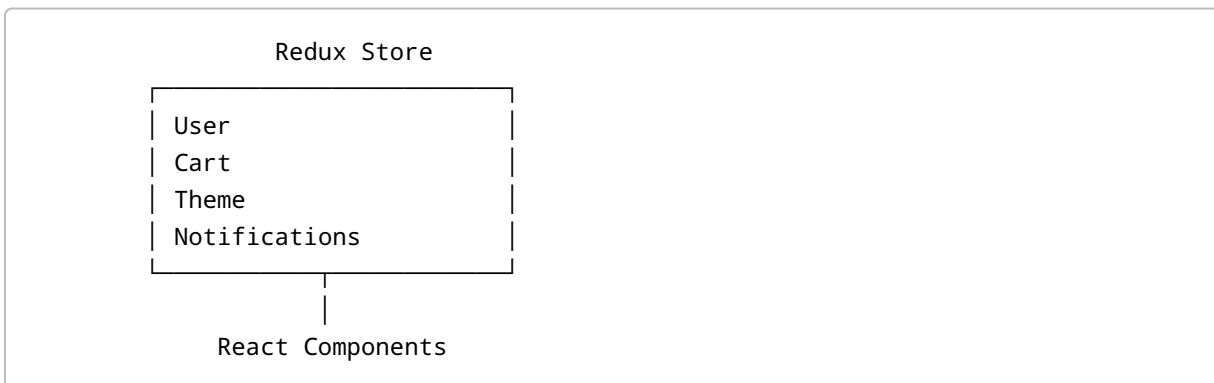
3. Redux

Purpose

Redux provides a **central store** for shared application state.

Real-world analogy

Think of the Redux Store as a **central warehouse**.



Instead of components independently managing complicated shared data, they interact with one central store.

Redux Flow

```

User clicks button
  ↓
Component
  ↓

```

```
dispatch(action)
  ↓
  Reducer
  ↓
New State
  ↓
Redux Store
  ↓
Components update
```

Important terms

Store

The central place where Redux state is kept.

Action

Describes what happened.

```
{ type: "cart/add", payload: product }
```

Reducer

Function that determines how state changes.

```
(state, action) → newState
```

Dispatch

Sends an action to Redux.

```
dispatch(addToCart(product));
```

Selector

Reads data from the store.

```
useSelector(state => state.cart.items);
```

4. Complete Redux Example

Install

```
npm install @reduxjs/toolkit react-redux
```

counterSlice.js

```
import { createSlice } from "@reduxjs/toolkit";

const counterSlice = createSlice({
  name: "counter",

  initialState: {
    value: 0
  },

  reducers: {
    increment: (state) => {
      state.value += 1;
    },

    decrement: (state) => {
      state.value -= 1;
    }
  }
});

export const { increment, decrement } = counterSlice.actions;

export default counterSlice.reducer;
```

store.js

```
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "./counterSlice";

export const store = configureStore({
  reducer: {
    counter: counterReducer
  }
});
```

The store now conceptually contains:

```
{
  counter: {
    value: 0
  }
}
```

main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import { Provider } from "react-redux";

import App from "./App";
import { store } from "./store";

ReactDOM.createRoot(document.getElementById("root")).render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

Provider makes the Redux store available to the React application.

App.jsx

```
import { useSelector, useDispatch } from "react-redux";
import { increment, decrement } from "./counterSlice";

function App() {
  const count = useSelector(
    state => state.counter.value
  );

  const dispatch = useDispatch();

  return (
    <>
      <h1>{count}</h1>

      <button onClick={() => dispatch(increment())}>
        +
      </button>
    </>
  );
}
```

```

        <button onClick={() => dispatch(decrement())}>
          -
        </button>
      </>
    );
  }

  export default App;

```

Complete flow

```

Click "+"
  ↓
dispatch(increment())
  ↓
Redux receives action
  ↓
increment reducer runs
  ↓
Store:
counter.value = 1
  ↓
useSelector sees new value
  ↓
Component re-renders

```

5. TanStack Query

Purpose

TanStack Query is mainly for **server state**.

Server state means data that lives outside your React application:

```

React App
  ↓
API
  ↓
Backend

```

↓
Database

Examples:

Users
Products
Orders
Comments
Analytics

Why do we need TanStack Query?

Without it, you might manually manage:

```
const [data, setData] = useState(null);  
const [loading, setLoading] = useState(false);  
const [error, setError] = useState(null);
```

Then write `useEffect()` and `fetch()`.

You also need to handle:

Loading
Error
Caching
Refetching
Retries
Stale data
Duplicate requests

TanStack Query handles much of this for you.

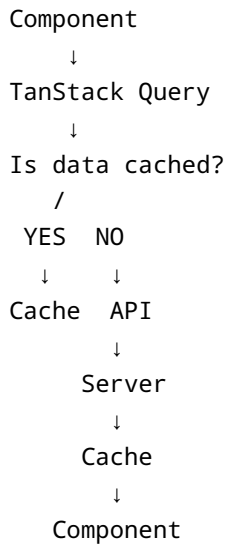
Real-world analogy

Think of a **library**.

You ask for:

"Give me the React book."

The librarian checks whether it is already available.



So TanStack Query is essentially a **smart manager for API/server data**.

6. useQuery

`useQuery()` is the main hook used to fetch/read server data.

The two most important properties are:

`queryKey`
`queryFn`

`queryKey`

Identifies the data.

`queryKey: ["products"]`

Think:

"This cache belongs to products."

queryFn

Function that actually gets the data.

```
queryFn: getProducts
```

Think:

"This is how I fetch the products."

7. Complete TanStack Query Example

Install

```
npm install @tanstack/react-query
```

main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";

import {
  QueryClient,
  QueryClientProvider
} from "@tanstack/react-query";

import App from "./App";

const queryClient = new QueryClient();

ReactDOM.createRoot(document.getElementById("root")).render(
  <QueryClientProvider client={queryClient}>
    <App />
  </QueryClientProvider>
);
```

`QueryClient` manages TanStack Query's cache and configuration.

`QueryClientProvider` makes it available to your React components.

App.jsx

```
import { useQuery } from "@tanstack/react-query";

async function getProducts() {
  const response = await fetch(
    "https://fakestoreapi.com/products"
  );

  if (!response.ok) {
    throw new Error("Failed to fetch products");
  }

  return response.json();
}

function App() {

  const {
    data,
    isLoading,
    isError,
    error
  } = useQuery({
    queryKey: ["products"],
    queryFn: getProducts
  });

  if (isLoading) {
    return <h2>Loading...</h2>;
  }

  if (isError) {
    return <h2>{error.message}</h2>;
  }

  return (
    <div>
      <h1>Products</h1>

      {data.map(product => (
        <div key={product.id}>
          <h3>{product.title}</h3>
          <p>${product.price}</p>
        </div>
      ))}
    </div>
  );
}
```

```
    );  
  }  
  
  export default App;
```

What happens?

```
App renders  
  ↓  
useQuery()  
  ↓  
Checks queryKey ["products"]  
  ↓  
Runs getProducts()  
  ↓  
API request  
  ↓  
Data received  
  ↓  
TanStack Query caches it  
  ↓  
data becomes available  
  ↓  
Component renders products
```

8. Why the `queryKey` matters

These are separate queries:

```
["products"]
```

```
["products", 5]
```

```
["products", 10]
```

Conceptually:

```
TanStack Query Cache
```

```
["products"]  
  ↓  
  All products  
  
["products", 5]  
  ↓  
  Product #5  
  
["products", 10]  
  ↓  
  Product #10
```

This lets TanStack Query know which cached data belongs to which request.

9. Redux vs TanStack Query

This is the most important comparison.

Redux

"State created/managed by my application."

Examples:

```
Cart  
Modal state  
Selected item  
UI preferences  
Complex client state
```

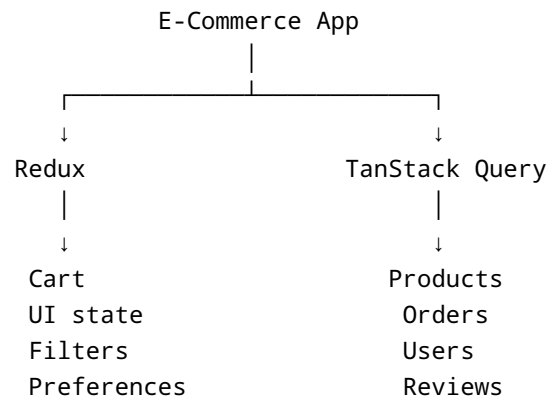
TanStack Query

"Data coming from my server."

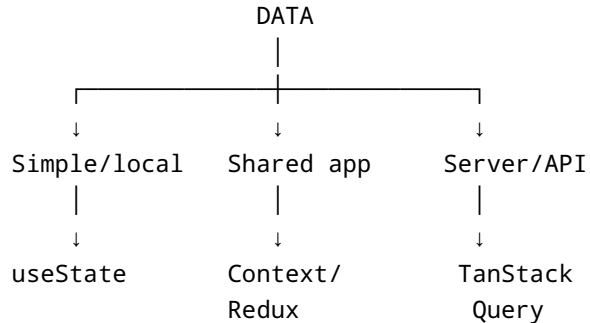
Examples:

```
Products  
Users  
Orders
```

Example e-commerce app



10. Context vs Redux vs TanStack Query



Context

"Make this value available to many components."

Redux

"Manage complex shared application state centrally."

TanStack Query

"Fetch, cache, synchronize and manage server data."

11. Final Cheat Sheet

useState

→ Local state

Context

→ Share values

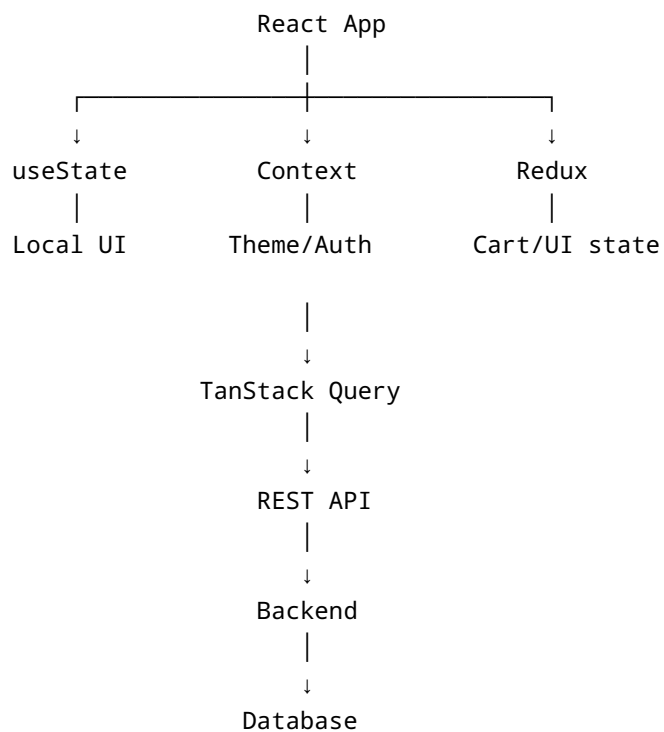
Redux

→ Centralized client/application state

TanStack Query

→ Server/API state

Typical React application



One-line memory trick

Context shares, Redux manages, TanStack fetches & caches.